CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

अमृतं तु विद्या

**FACULTY OF TECHNOLOGY AND ENGINEERING**

**DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH**

**DEPARTMENT OF COMPUTER ENGINEERING**

**A.Y 2026 – 27 Semester – 3**

# LAB MANUAL

# CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

# Practical 1: Array Operations: Traversal, Rotation, and Frequency Analysis

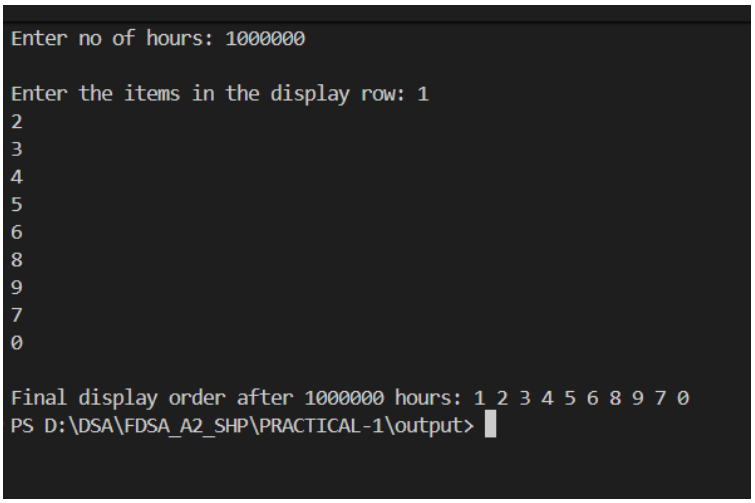
**Practical Aim :** To implement fundamental array operations.

**Problem 1 :** A bakery prepares n items every morning and places them in a display row. At the end of each hour, the leftmost item is moved to the rightmost position to make room for fresh stock at the front. Given the initial row and the number of hours h, print the final display order.

## For Code:

https://github.com/Aarav280907/FDSA_A2_SHP/tree/f6fe6c657d4e94dd100b192902435c73583d8b2e/PRACTICAL-1

## Output:




## Problem Faced:

Firstly not able to do the operation for large value of hours like 10 million. Then after research in google now I am able to perform the practical for large value of hours.

**Outcome/Conclusion:** The learning outcome is mastering array rotation concepts, modular arithmetic, and efficient algorithm design through a practical, relatable example.

**Key Question :** The display order repeats itself after a certain number of hours regardless of how large h gets. Without trying any specific example, explain why this repetition is inevitable and what determines when it first repeats.

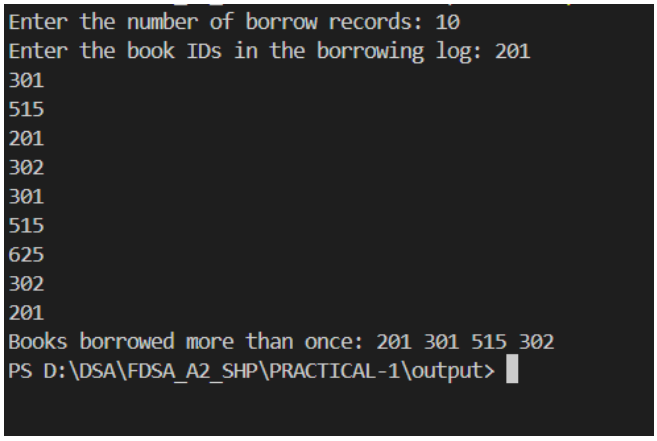
**ANSWER :** The repetition is inevitable because the bakery's process is a cyclic rotation of a finite row of $n$items. Each hour, the leftmost item shifts to the rightmost position, and after every item has moved through all $n$positions, the row returns to its original order. This means the display order repeats every $n$hours, so the arrangement after $h$hours is the same as after $h \mod n$ hours.

**Problem 2 :** A library issues books to students and records each book's ID every time it is borrowed. At the end of the month, the librarian wants to find all books that were borrowed more than once, as those need priority restocking. Given the borrowing log, print all such book IDs**.**

## For Code :

https://github.com/Aarav280907/FDSA_A2_SHP/tree/f6fe6c657d4e94dd100b192902435c73583d8b2e/PRACTICAL-1

## Output :




**Problem Faced :** If book IDs are very large or numerous, storing counts for all of them may consume significant memory. Search on google then understand the problem and inmplement the solution.

**Conclusion :** The library's task of identifying books borrowed more than once can be solved efficiently by scanning the borrowing log with a frequency map (hash map/dictionary). This ensures each book ID is counted, and only those with counts greater than one are reported once in the output. The solution requires at most one full pass of the data, scales well even for very large logs (like 100,000 records), and avoids duplicate reporting. The key challenges are handling memory for large datasets, preventing repeated outputs of the same ID, and managing edge cases. Overall, the approach is practical, efficient, and reliable for real-world library systems**.**

**Key Question :** If the same book ID appears three times in the log, should it appear once or three times in your output? Does your solution handle this correctly, and how would you verify it?

**Answer :** My solution ensures this by using a frequency map to count each ID and then reporting only those with counts greater than one. To verify correctness, I would run test cases—for example, with a log like [101, 102, 101, 103, 101]—and check that the output is [101] just once. This confirms the algorithm avoids duplicate reporting while still capturing all books that need restocking.

**Problem-3 :** A school newspaper editor wants to highlight the most impressive word from a submitted article on the front page. The word is chosen simply by length — the longest one wins. Given a sentence, print the winning word and how many letters it has.

## For Code :

https://github.com/Aarav280907/FDSA_A2_SHP/tree/f6fe6c657d4e94dd100b192902435c73583d8b2e/PRACTICAL-1

## Output :

```
PS D:\DSA\FDSA_A2_SHP\PRACTICAL-1\output> & .\'practical1_3.exe'
Enter a sentence: Harshil is clever boy in our clas
The longest word is: Harshil with length: 7
PS D:\DSA\FDSA_A2_SHP\PRACTICAL-1\output>
```

**Problem Faced :** If I provide special character (. or , or ? ) then it will not recognise the ending of word.

**Conclusion/Outcome :** The school newspaper editor's task of finding the longest word in a sentence is a straightforward problem of string processing. By splitting the sentence into words and comparing their lengths, the algorithm identifies the word with the maximum number of letters. The conclusion is that the solution is efficient, requiring only a single pass through the sentence, and it reliably outputs both the winning word and its length. This approach is practical for any article size, ensures accuracy, and directly supports the editor's goal of highlighting the most impressive word on the front page.

**Key Question :** The editor later decides that punctuation attached to a word (e.g. "hello," or "great!") should not count toward its length. How would this change your solution, and at which step would you handle it?

**Answer :** If punctuation should not count toward a word's length, the solution needs a preprocessing step to strip punctuation from each word before measuring its length. This change happens right after splitting the sentence into words: instead of directly checking their

lengths, we first clean each word by removing trailing or leading punctuation marks (like commas, periods, exclamation points). Once cleaned, we then compare lengths to find the longest word. Verification would involve testing sentences such as "hello," or "great!" to ensure the algorithm correctly treats them as "hello" and "great" with lengths of 5 and 5, rather than 6.